AI-Powered Log Analysis & Incident Triage System

Project Proposal for UCS503P
Submitted to: Dr. Jeelani Asif
Thapar Institute of Engineering and Technology

Arnav Singh, CSED* Siddharth Chandra, CSED†

August 12, 2026

1 Higher-order goal

This project aims to improve modern DevOps and software system reliability by drastically reducing the time and manual effort required to diagnose application failures. While system monitoring spans vast cloud infrastructures, the immediate engineering objective is to translate automated log parsing and LLM-driven root cause analysis (RCA) into a measurable, deployable, and developer-centric software platform.

2 Time-to-value

We will deliver meaningful increments quickly by prototyping the core log ingestion and triage workflow and validating it within a 10-week Agile development window across 2-week Sprint iterations.

We will prioritize fast feedback over perfection by utilizing bi-weekly Sprint reviews and automated CI/CD pipelines early to eliminate release friction. Success is defined by what can be delivered and measured in the initial iteration, then iteratively expanded based on system telemetry and user testing.

3 Problem Statement

Software engineering and Site Reliability Engineering (SRE) teams managing distributed microservices face severe delays and operational fatigue during system outages. Common pain points include:

- **Fragmentation:** Log data is scattered across multiple services, containers, and isolated text files, causing lost context during incidents.
- **Low Transparency & High Noise:** Standard logging tools display thousands of cryptic, raw stack traces without explaining the actual failure trigger or severity.
- **Slow Incident Coordination:** On-call engineers lack a unified, automated triage workflow, spending up to 70% of incident response time manually tracing root causes.

*Roll No: 1024160011, Email: asingh30_be24@thapar.edu
†Roll No: 1024160017, Email: schadra_be24@thapar.edu

- **Alert Fatigue:** High volume and duplicate error streams flood alerting channels, leading to missed critical failures and extended Mean Time to Resolution (MTTR).

Why this matters now: In modern high-availability applications, even minor downtime incidents incur significant financial losses; reducing diagnosis time directly protects service reliability.

4 Proposed Solution

4.1 Overview

Build a web-based **AI-Powered Log Analysis & Incident Triage System** comprising the following components:

- **Log Ingestion Engine:** High-throughput API pipeline to ingest, parse, and index structured and unstructured log batches in real time.
- **Automated AI Triage & RCA:** LLM-driven error classification, stack trace translation, and root-cause analysis (RCA) generation.
- **Incident Management Workflow:** Real-time stream filtering, error velocity tracking, and incident status updates (`Open` → `Investigating` → `Resolved`).
- **Real-Time Dashboard & Alerts:** WebSocket-powered dashboard featuring error rate charts and automated webhook alerts (Slack/Discord).

4.2 Core Workflow

1. Microservices emit log payloads to the central ingestion REST endpoint.
2. The ingestion pipeline parses incoming stack traces, performs caching/deduplication in Redis, and indexes entries in PostgreSQL/Elasticsearch.
3. For anomalous error spikes or critical exceptions, the system triggers the LLM inference engine to synthesize a plain-language RCA report and suggested remediation steps.
4. Engineers review the live dashboard, receive automated notifications, and update incident statuses upon resolution.

4.3 Operational Constraints

- Keep the dashboard UI responsive and intuitive on standard web browsers.
- Focus heavily on robust workflow engineering, stream processing, caching, and system correctness rather than training custom foundational LLMs.
- Design for containerized deployment using standard platform hosting and managed databases.

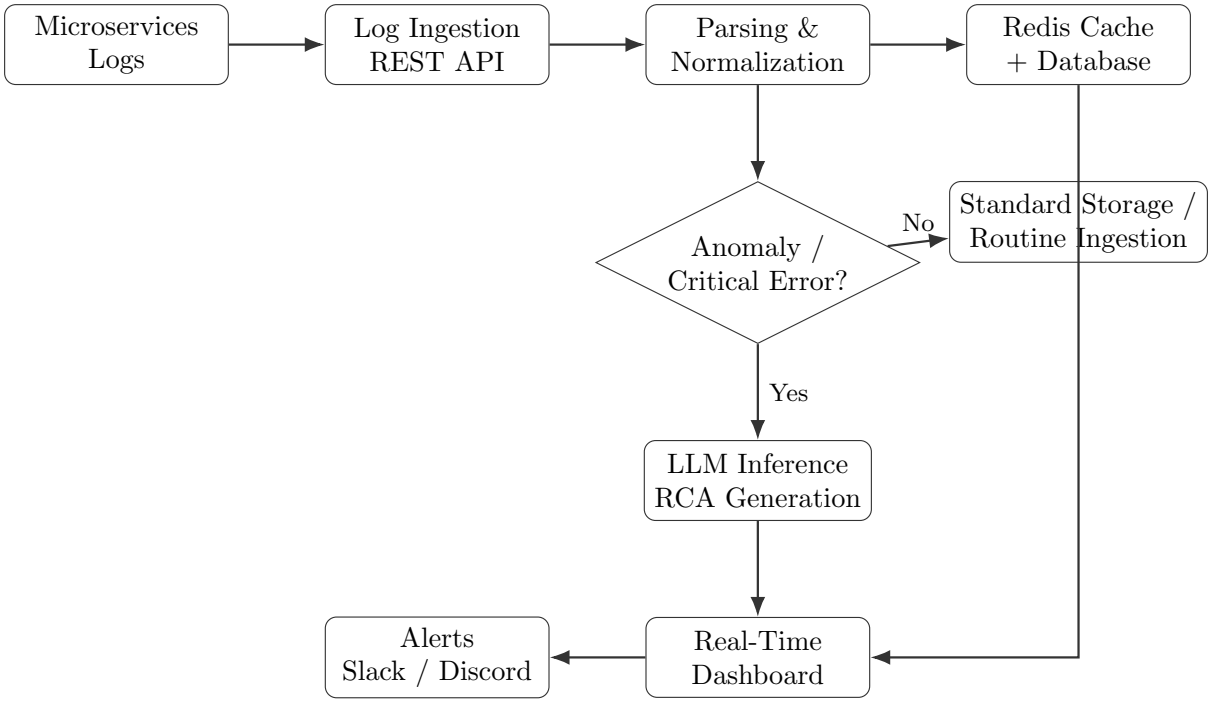

Figure 1: End-to-End Log Analysis and Incident Triage Workflow

5 Solution Approach

5.1 AI & Deduplication Assistance

Duplicate log handling and root cause analysis will rely on heuristic and inference-driven strategies:

- Normalize incoming log messages by stripping dynamic parameters (e.g., timestamps, memory addresses).
- Use Redis caching to store generated AI analyses for identical error signatures, preventing redundant API calls during error storms.
- Present LLM-generated RCA summaries and suggested fixes directly to engineers to assist human decision-making.

5.2 System Architecture

A modern 3-tier decoupling model:

- **Frontend:** Responsive React dashboard for real-time log tailing, error velocity visualizer, and incident inspection.
- **Backend API:** FastAPI (Python) web service handling REST ingestion, WebSocket connections, LLM prompt orchestration, and alert triggers.
- **Data & Cache Layer:** PostgreSQL/Elasticsearch for persistent query indexing, and Redis for rate-limiting and response caching.

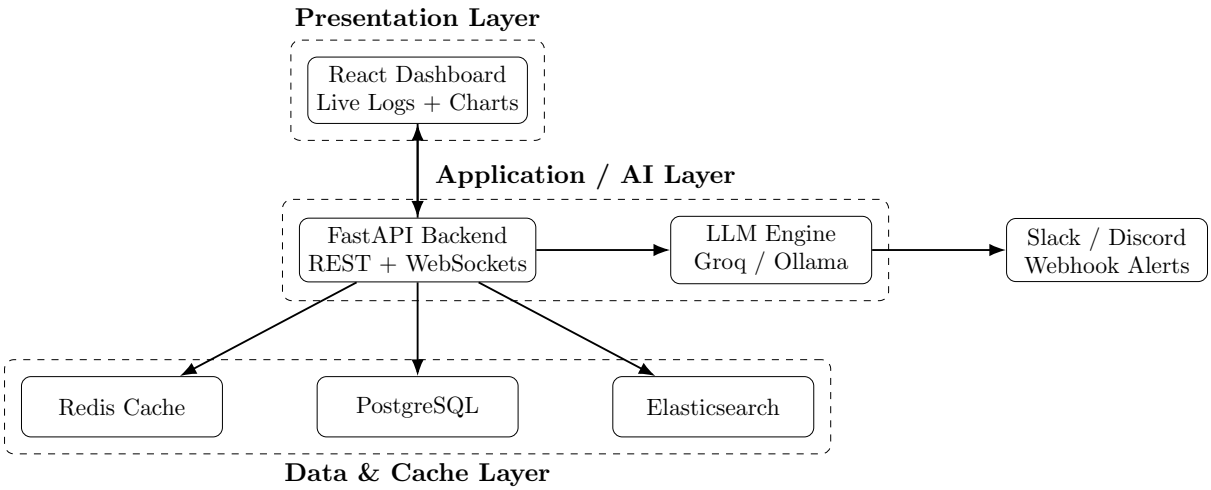

Figure 2: Three-Tier System Architecture

5.3 CI/CD and Fast Delivery Enabler

CI/CD pipelines will be configured to:

- Run automated linting, unit tests, and integration tests on every commit.
- Produce containerized Docker artifacts for repeatable staging and production deployments.
- Enable seamless, incremental releases across each Agile Sprint.

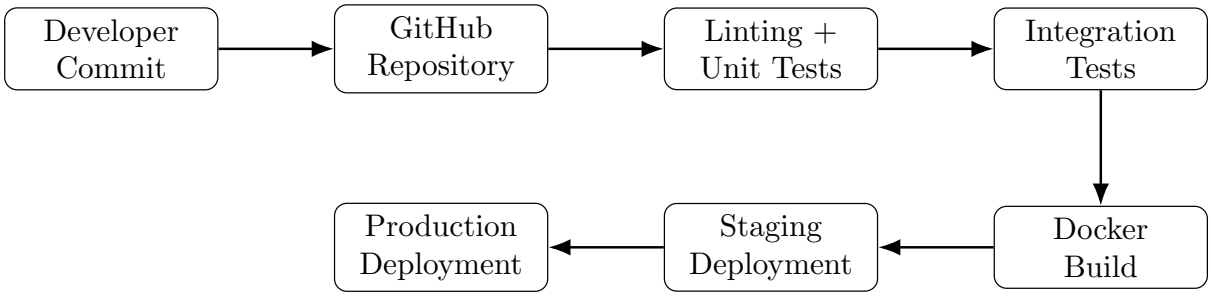

Figure 3: Automated CI/CD Pipeline

6 Evaluation Criteria

6.1 Primary Evaluation Metric

Mean Time to Diagnosis (MTTD): The duration between a critical log error entering the system and an automated RCA report being generated and displayed.

- *Target:* Median MTTD < 5 seconds during stress testing.
- *Attribution:* Measured via backend processing timestamps (ingestion log time vs. LLM output completion time).

6.2 Secondary Evaluation Metrics

- **Log Ingestion Throughput:** Sustained logs processed per second without dropped connection payloads.
- **Cache Hit Rate:** Percentage of duplicate error traces served via Redis without hitting external LLM endpoints.
- **System Uptime & Reliability:** Availability of dashboard and API ingestion endpoints during test runs ($> 99\%$ operational target).

7 Scalability & Agile Delivery Roadmap

7.1 Scalability Considerations

- **Theoretically:** Decoupled architecture using asynchronous event loops in FastAPI, stateless REST routes, and indexed database lookups to handle heavy log traffic spikes.
- **Practically:** Containerized deployment (Docker Compose), managed cloud databases, and Redis caching layers.

7.2 10-Week Agile Sprint Deliverables

- **Weeks 1–2 (Sprint 1 — Core Ingestion & CI Setup):**
  - Setup repository, CI/CD pipeline, and Docker environments.
  - Implement baseline FastAPI log ingestion endpoint and database schemas.
- **Weeks 3–4 (Sprint 2 — Search, Indexing & Storage):**
  - Integrate indexing layer (PostgreSQL/Elasticsearch) for rapid filtering.
  - Build synthetic microservice log generator for load testing.
- **Weeks 5–6 (Sprint 3 — LLM Integration & Redis Caching):**
  - Integrate Groq/Ollama API for automated stack trace translation and RCA generation.
  - Implement Redis caching to prevent duplicate LLM calls.
- **Weeks 7–8 (Sprint 4 — Real-time UI & WebSockets):**
  - Build React dashboard with real-time log streams over WebSockets.
  - Integrate webhook alerting engine for Slack/Discord.
- **Weeks 9–10 (Sprint 5 — Polish, Deployment & Evaluation):**
  - Deploy system to staging/production hosting (Render/Vercel).
  - Conduct benchmark stress testing and complete final documentation.

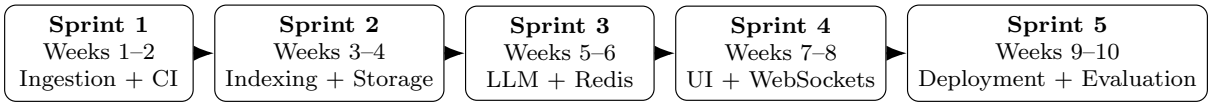

Figure 4: 10-Week Agile Development Roadmap

8 Engine availability heuristic

A mature set of "engines" (tooling/services) is available to implement this as a web-based project:

- **Web framework:** FastAPI (Python) for asynchronous REST log ingestion and WebSocket streaming.
- **Managed relational / search database:** PostgreSQL for structured incident storage and Elasticsearch for high-performance log querying and indexing.
- **Caching & rate-limiting engine:** Redis for in-memory signature caching, response reuse, and WebSocket session management.
- **AI inference engine:** Groq API (Llama 3 inference) or local Ollama instances for low-latency stack trace translation and Root Cause Analysis (RCA).
- **Test framework:** `pytest` and `httpx` for unit, schema validation, and API integration testing.
- **CI runner and staging deployment target:** GitHub Actions for automated linting/tests and Render / Vercel containerized staging environments.

We will use these mature components to focus on workflow design, caching logic, and measurement rather than inventing new infrastructure.

9 Project scope and deliverables

9.1 Initial deliverable

- Ingestion REST endpoint to receive structured JSON and raw text log streams.
- Database schema setup (PostgreSQL) with automated timestamp logging for MTTD measurement.
- Synthetic mock log generator script simulating microservice errors (database connection drops, payment timeouts).
- CI: build + backend unit tests + linting via GitHub Actions.
- CD to staging environment via a single containerized pipeline job.

9.2 Subsequent deliverables

- Automated LLM-driven error parsing and plain-language RCA generation.
- Redis caching layer to suppress duplicate AI API calls during high-frequency error spikes.
- Real-time React dashboard with live WebSocket log streaming and error velocity charts.
- Incident triage workflow (`Open` → `Investigating` → `Resolved`) with webhook notifications to Discord/Slack.
- Benchmark evaluation suite measuring sustained log throughput and cache hit rate under load.

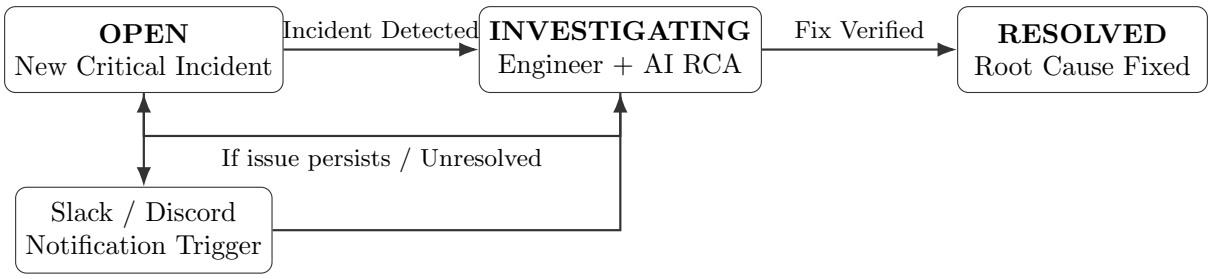

Figure 5: Incident Triage and Resolution Workflow

10 Risks and Mitigations

- **API Rate Limits / Cost Spikes:** Mitigate by using local LLMs (Ollama) or free-tier Groq APIs backed by aggressive Redis caching.
- **Log Ingestion Backpressure:** Mitigate by using asynchronous background tasks and queueing log streams to avoid blocking main threads.
- **Data Privacy / Sensitive Information:** Implement regex maskers on the ingestion pipeline to strip passwords, API keys, and PII prior to storage or LLM processing.

11 Summary

This proposal details an actionable, engineering-focused software project designed for a 10-week Agile development cycle. By prioritizing rapid time-to-value, automated CI/CD releases, clear metrics (MTTD), and practical full-stack AI integration, the project directly solves real-world DevOps triage delays while delivering a high-impact, production-grade resume piece.